

# 1C Accounting Automation Platform — Technical Design

Oct 4, 2026 · @Marshall

## 1. Overview

The platform removes manual data entry from accounting in Uzbekistan: data flows from the tax portal, EDO operators and banks straight into 1C, checked and correct, and an AI assistant audits the books.

**Product (working name: TBD).** Three systems:

1. **Desktop app (.exe)** on the accountant's PC: pulls external data, processes it, and writes correct documents into 1C.
2. **Control system:** user accounts, licenses, automatic billing, AI usage metering, admin dashboard.
3. **Marketing website:** product pages, pricing, ordering and payment.

**Primary user.** An accountant who keeps the books of several companies in 1C (one PC, several infobases). The owner's own five bases are the first production users.

**Target platform.** 1C:Enterprise 8.3, configuration *Бухгалтерия для Узбекистана*, редакция 3.0.

### Success criteria for the MVP

- An e-invoice received or issued on Didox appears in 1C as a correct, unposted Счет-фактура in under 1 minute, with no typing.
- A bank statement for a day is loaded into 1C with every line matched to a counterparty and account, or flagged for review.
- The audit report finds every mismatch between Didox, bank and 1C for a chosen period.
- A new customer can register, pay and activate the desktop app without contacting us.

## 2. Scope

Phase 1 (MVP) covers the flows with reliable data sources and immediate value; everything else follows once the core 1C write path is proven.

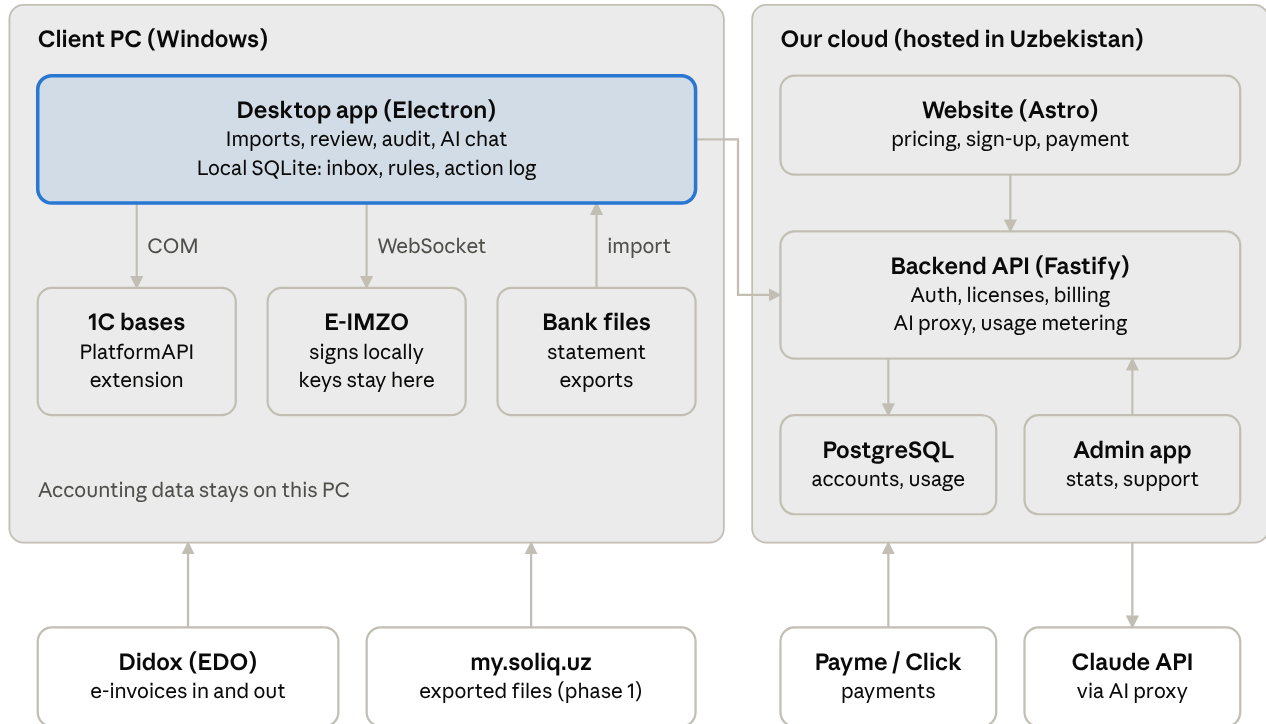
| Area           | Phase 1 (MVP)                                                               | Phase 2                                                    | Phase 3                                       |
|----------------|-----------------------------------------------------------------------------|------------------------------------------------------------|-----------------------------------------------|
| 1C connector   | Read metadata and catalogs; create unposted Счет-фактура and bank documents | Post documents after approval; more document types         | Costing documents, production                 |
| Invoices       | Import from Didox (issued and received)                                     | Soliq-service, other EDO operators; issue invoices from 1C | Auto-matching to contracts                    |
| Bank           | Import 1CClientBankExchange files                                           | Direct APIs of the top 3 banks                             | Payment orders from 1C to bank                |
| Tax portal     | Manual export/import of my.soliq.uz files                                   | Automated pull if a legal API route exists                 | Tax report pre-checks                         |
| Audit          | Rule-based reconciliation report                                            | AI explanations and fix suggestions                        | Continuous monitoring                         |
| AI             | Chat assistant over 1C data (read-only)                                     | AI account mapping for unknown lines                       | AI drafting of documents                      |
| Costing        | —                                                                           | —                                                          | Production costing (материалы, полуфабрикаты) |
| Birdarcha      | —                                                                           | Integration (scope to confirm)                             | —                                             |
| Control system | Accounts, licenses, manual + Payme billing, AI metering                     | Click, Uzum; subscriptions; usage dashboard                | Partner/reseller accounts                     |
| Website        | Landing, pricing, register, pay                                             | Docs, blog, case studies                                   | —                                             |

**Non-goals for phase 1:** posting documents automatically, 1C configurations other than БУ для Узбекистана 3.0, macOS/Linux desktop, mobile apps, multi-user collaboration inside one firm.

### 3. System architecture

The desktop app does all work with company data on the client's PC; our cloud only handles accounts, licenses, payments and AI requests.

**Books stay on the client PC; our cloud handles accounts, money and AI**



system architecture · client PC, our cloud, 4 external services

The app writes to 1C over COM and signs through the local E-IMZO client; its only link to our cloud is the license check, updates and the AI proxy, so the books never reach our servers.

### 4. Desktop app

The desktop app is an Electron application (TypeScript, Vite, React) that runs on Windows 10/11 next to 1C and does all work with company data locally.

#### Process model

- **Main process (Node):** talks to 1C, E-IMZO, bank files, Didox and the backend; runs the sync jobs and the audit engine; stores local data.
- **Renderer (React UI):** screens only; reaches the main process through a typed IPC bridge ( `contextBridge` , no `nodeIntegration` ).

- **Background jobs:** a job queue in the main process (one worker per infobase) so a slow 1C call never freezes the UI.

## Screens (phase 1)

| Screen          | Purpose                                                                                             |
|-----------------|-----------------------------------------------------------------------------------------------------|
| Sign in         | Log in with the platform account; activate the license on this PC                                   |
| Companies       | List of connected infobases: name, INN, 1C path, connector status, last sync                        |
| Connect company | Pick the infobase, install/check the 1C extension, link the Didox account, set the bank file folder |
| Inbox           | Incoming items (invoices, bank lines) with status: new, matched, needs review, written to 1C, error |
| Review item     | Side-by-side: source data vs the 1C document to be created; edit mapping; approve                   |
| Audit           | Run a reconciliation for a period; see findings; export to Excel                                    |
| AI assistant    | Chat over the selected company's 1C data (read-only tools)                                          |
| Settings        | E-IMZO status, language (uz / ru / en), updates, logs                                               |

**Local storage.** SQLite ( `better-sqlite3` ) per user profile: companies, sync state, inbox items, mapping rules, audit runs, the action log. Secrets (Didox tokens, connector tokens) go to the Windows Credential Manager via `keytar` / `safeStorage` , never into SQLite in plain text.

**Updates.** `electron-updater` with signed releases from our update server. A code-signing certificate is required so Windows SmartScreen does not block the installer.

**Licensing behavior.** The app checks its license with the backend at start and every 6 hours. Offline, it keeps working for a 7-day grace period on a signed license token, then switches to read-only mode.

## 5.1C connector

All reading and writing of 1C data goes through our own configuration extension (.cfe); the desktop app calls it, never the 1C screens.

**Why an extension.** It installs without changing the main configuration, survives 1C updates, and keeps the document-building logic in BSL where 1C's own rules (filling, posting checks) apply.

### Extension contents

- Common module `PlatformAPI` (server, flag *Внешнее соединение* on): exported functions that take and return JSON strings.
- HTTP service `platform` with the same functions as endpoints (for published infobases, phase 2).
- Catalog `PlatformLog`: one record per write (source, external id, created 1C document, user, time).
- Attribute `ExternalID` on the documents we create, so a repeated import never makes a duplicate.

**Transport (phase 1): COM external connection.** The desktop app opens

`V83.COMConnector` → `Connect("File=\"D:\\...\";Usr=...;Pwd=...")` and calls `PlatformAPI` functions. It works without 1C being open and without publishing the base on a web server. Requirements: `comcntr.dll` registered, platform bitness = app bitness (64-bit). From Node: the `winax` native module, rebuilt for Electron. Each connection uses a 1C license; the app keeps one connection per infobase and closes it when idle.

### Functions (phase 1)

| Function                        | Input                            | Output                                                    |
|---------------------------------|----------------------------------|-----------------------------------------------------------|
| <code>Ping</code>               | —                                | config name, version, platform version, extension version |
| <code>GetOrganizations</code>   | —                                | organizations with INN                                    |
| <code>FindCounterparty</code>   | INN or name                      | matches with ref                                          |
| <code>CreateCounterparty</code> | INN, name, address, bank account | ref                                                       |
| <code>FindItem</code>           | name, IKPU code, unit            | matches with ref                                          |

| Function              | Input                           | Output                          |
|-----------------------|---------------------------------|---------------------------------|
| CreateInvoiceReceived | invoice JSON (header + lines)   | document ref, number; unposted  |
| CreateInvoiceIssued   | invoice JSON                    | document ref, number; unposted  |
| CreateBankDocuments   | statement lines                 | refs per line; unposted         |
| GetBalances           | account, period                 | turnover and balances           |
| RunQuery              | query text + params (read-only) | rows (for the audit and the AI) |
| GetLog                | period                          | our write log                   |

**Rules for every write:** documents are saved unposted (*записать без проведения*); ExternalID is checked first; the call runs in one transaction; errors return a structured code + message, never a 1C exception text alone.

**Document mapping.** Exact object and field names in БУ для Узбекистана 3.0 are to be read from the live base with `get_metadata` before coding (Счет-фактура полученный/выданный, Поступление/Списание с расчетного счета, Контрагенты, Номенклатура, Договоры). The mapping lives in one file per configuration version so a 1C update means updating one map.

## 6. External integrations

Every integration sits behind one TypeScript interface, so a source can be swapped or added without touching the inbox, mapping or 1C code.

```
interface DocumentSource {
  id: 'didox' | 'soliq-service' | 'soliq-file' | 'bank-file' | 'bank-api';
  connect(company: Company): Promise<ConnectionStatus>;
  fetch(company: Company, period: Period): Promise<SourceItem[]>; //
  normalized
}
```

| Source                       | What we get                                                  | Method                                                                                                    | Phase     | To verify                                                                 |
|------------------------------|--------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------|-----------|---------------------------------------------------------------------------|
| Didox                        | Issued and received e-invoices (счета-фактуры), acts, status | Didox API with the company's account; auth signed through E-IMZO                                          | 1         | API docs, partner access, rate limits, test environment                   |
| Soliq-service (EDO operator) | Same document types as Didox for companies on that operator  | Operator API                                                                                              | 2         | Whether a public/partner API exists                                       |
| my.soliq.uz                  | Income/expense registers, tax account data                   | Phase 1: the user exports files, the app imports them. Phase 2: automation only through an official route | 1 (files) | Is there an official API? Scraping is not planned (fragile, terms of use) |
| Banks                        | Statement lines (payments in/out)                            | Phase 1: statement export files (1CClientBankExchange or each bank's TXT/XML). Phase 2: bank APIs         | 1 (files) | Export formats of the banks our first users use                           |
| Birdarcha                    | To confirm                                                   | To confirm                                                                                                | 2         | Exact site and purpose (open question)                                    |
| E-IMZO                       | Digital signature of auth requests and documents             | Local E-IMZO client on the user's PC, called over its local WebSocket                                     | 1         | API key requirements for a desktop (non-browser) origin                   |

**Normalized item.** Every source produces the same `SourceItem` type (invoice in/out, bank in/out), external id, date, number, counterparty INN and name, contract, lines (item, IKPU code, unit, qty, price, VAT rate, VAT sum, total), raw payload for the audit trail.

**Signing rule.** E-IMZO keys stay on the user's PC. The app asks the local E-IMZO client to sign; the private key and its password are never read, stored or sent by our code.

## 7. Processing modules

Invoices and bank lines go through one pipeline; nothing reaches 1C without passing matching and, where unsure, a human review.

### Import pipeline

1. **Fetch** items from a source for a company and period.
2. **Deduplicate** by external id against the local store and `PlatformLog` in 1C.
3. **Match references**: counterparty by INN (create if missing, after review), item by IKPU code then name, contract by counterparty + date.
4. **Map accounts**: apply mapping rules (per company, per counterparty, per item group) to pick счета учета and статьи; an unknown line goes to review.
5. **Review**: items with confidence below the threshold, new counterparties or sums over a limit wait in the Inbox for approval.
6. **Write** to 1C through the connector, unposted, with `ExternalID`.
7. **Log** the result locally and in `PlatformLog`.

**Mapping rules.** Stored per company as ordered rules: `if counterparty = X and item group = Y → account Z`. Every manual fix in Review can be saved as a new rule, so the system learns the accountant's choices.

### Audit engine (phase 1, rule-based)

| Check              | Compares                                                    |
|--------------------|-------------------------------------------------------------|
| Missing invoices   | Didox invoices vs Счета-фактуры in 1C, by number, date, INN |
| Sum / VAT mismatch | Totals and VAT per invoice, source vs 1C                    |
| Missing bank lines | Bank statement vs bank documents in 1C                      |
| Negative balances  | Cash (касса) and bank accounts below zero on any day        |
| Unposted documents | Documents saved but never posted in the period              |
| Counterparty data  | INN, name or bank account differing between source and 1C   |
| VAT reconciliation | Input/output VAT per Didox vs VAT accounts in 1C            |

Output: a findings list with severity, a link to the 1C document (navigation link) and an Excel export.

**AI assistant (phase 1, read-only).** Chat in the desktop app; tools = `RunQuery` , `GetBalances` , metadata, audit findings. Model calls go through our backend (section 8), never with an API key in the app. Phase 2 adds AI suggestions for unknown account mappings and plain-language explanations of audit findings.

**Costing (phase 3).** Production costing for manufacturing clients: materials and semi-finished goods, overhead allocation bases, cost per unit and per batch. Requirements to be written with the first manufacturing customer.

## 8. Control system

The control system is one Node.js backend (Fastify, TypeScript, PostgreSQL) with an admin web app; it owns accounts, licenses, money and AI access, and never stores 1C accounting data.

### Backend modules

| Module           | Responsibility                                                                                                            |
|------------------|---------------------------------------------------------------------------------------------------------------------------|
| Auth             | Registration, login (email/phone + password, OTP by SMS), JWT access + refresh tokens, password reset                     |
| Accounts         | Customer account → users → connected companies (INN only, no books)                                                       |
| Plans & licenses | Plans (seats, companies, AI quota), device activation (max devices per seat), signed license tokens for offline grace     |
| Billing          | Invoices to customers, payments via Payme (phase 1), Click and Uzum (phase 2), auto-renewal, grace and suspension         |
| AI proxy         | Receives AI requests from the app, checks license and quota, calls the Claude API with our key, counts tokens per account |
| Usage metering   | Daily usage per account: companies, imports, documents written, AI tokens                                                 |

| Module    | Responsibility                                                    |
|-----------|-------------------------------------------------------------------|
| Releases  | Desktop app versions and the update feed                          |
| Admin API | Everything the admin dashboard needs; role-based (owner, support) |
| Audit log | Every admin action and every billing event                        |

## Billing logic

1. A customer picks a plan on the website and pays (Payme checkout).
2. The payment webhook marks the invoice paid and extends the license to the new end date.
3. Seven and one day before the end, the system sends reminders (SMS/email/Telegram bot) with a payment link.
4. No payment by the end date → 3-day grace → license suspended → the desktop app goes read-only. Data in 1C is never touched.
5. AI usage above the plan quota is billed as an add-on or blocked, per plan setting.

**AI proxy rules.** The app sends the conversation and tool results; the proxy adds the system prompt, picks the model (default Sonnet 5.5, Haiku 4.5 for simple steps), streams the answer back, and records input, output and cache tokens per request. Per-account daily and monthly caps protect us from runaway costs.

## Admin dashboard (Vite + React)

- Customers: list, plan, status, renewal date, companies count, last activity.
- Revenue: MRR, payments by method, overdue accounts.
- Usage: active users per day, imports and documents per day, AI tokens and cost per account vs revenue per account.
- Support tools: extend a license, reset devices, view an account's errors (from app telemetry, no accounting data).

## 9. Marketing website

The website sells the product and turns a visitor into a paying, activated customer without a sales call.

**Pages (phase 1):** Home (problem, how it works, demo video), Features, Pricing, Security (where data lives, E-IMZO keys stay local), FAQ, Download, Sign up / Log in, Customer cabinet (plan, invoices, devices, pay), Terms and Privacy. Languages: Uzbek (Latin), Russian; English later.

**Ordering flow**

- 1. Visitor picks a plan on Pricing.
- 2. Signs up (phone + SMS code).
- 3. Pays through Payme checkout (backend creates the invoice).
- 4. Gets the download link and signs in to the desktop app with the same account.
- 5. A 14-day trial plan skips step 3 (length to confirm).

**Build.** Astro (static pages, good SEO) with React islands for sign-up and the cabinet, served from the same backend domain family ( `www.` site, `api.` backend, `admin.` dashboard). Analytics: page views and the sign-up funnel.

10. Data model

The backend holds who pays and what they may use; the desktop holds the work. Accounting data stays on the customer’s PC and in their 1C.

**Backend (PostgreSQL)**

| Table         | Key fields                                                                                                   |
|---------------|--------------------------------------------------------------------------------------------------------------|
| accounts      | id, name, type (firm / company), status, created_at                                                          |
| users         | id, account_id, phone, email, password_hash, role (owner, member), last_login_at                             |
| plans         | id, code, price_uzs, period, seats, max_companies, ai_token_quota, features (jsonb)                          |
| subscriptions | id, account_id, plan_id, status (trial, active, grace, suspended, cancelled), starts_at, ends_at, auto_renew |
| devices       | id, user_id, machine_id (hash), name, activated_at, last_seen_at, revoked                                    |

| Table                    | Key fields                                                                                                                                                                                                                        |
|--------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>licenses</code>    | <code>id</code> , <code>subscription_id</code> , <code>device_id</code> , <code>token_jti</code> , <code>issued_at</code> , <code>expires_at</code>                                                                               |
| <code>companies</code>   | <code>id</code> , <code>account_id</code> , <code>inn</code> , <code>name</code> , <code>connected_at</code> (no accounting data)                                                                                                 |
| <code>invoices</code>    | <code>id</code> , <code>account_id</code> , <code>amount_uzs</code> , <code>status</code> , <code>due_at</code> , <code>paid_at</code> , <code>provider</code>                                                                    |
| <code>payments</code>    | <code>id</code> , <code>invoice_id</code> , <code>provider</code> (payme, click, uzum), <code>provider_txn_id</code> , <code>amount_uzs</code> , <code>state</code> , <code>raw</code> (jsonb)                                    |
| <code>ai_usage</code>    | <code>id</code> , <code>account_id</code> , <code>user_id</code> , <code>model</code> , <code>input_tokens</code> , <code>output_tokens</code> , <code>cache_read_tokens</code> , <code>cost_usd</code> , <code>created_at</code> |
| <code>usage_daily</code> | <code>account_id</code> , <code>date</code> , <code>imports</code> , <code>documents_written</code> , <code>ai_requests</code>                                                                                                    |
| <code>releases</code>    | <code>version</code> , <code>channel</code> , <code>url</code> , <code>sha256</code> , <code>notes</code> , <code>published_at</code>                                                                                             |
| <code>admin_audit</code> | <code>id</code> , <code>admin_id</code> , <code>action</code> , <code>target</code> , <code>payload</code> , <code>created_at</code>                                                                                              |

## Desktop (SQLite, local)

| Table                      | Key fields                                                                                                                                                                                                                               |
|----------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>companies</code>     | <code>id</code> , <code>backend_company_id</code> , <code>inn</code> , <code>name</code> , <code>infobase_path</code> , <code>extension_version</code> , <code>didox_linked</code> , <code>bank_folder</code>                            |
| <code>sync_state</code>    | <code>company_id</code> , <code>source</code> , <code>last_synced_at</code> , <code>cursor</code>                                                                                                                                        |
| <code>inbox_items</code>   | <code>id</code> , <code>company_id</code> , <code>source</code> , <code>external_id</code> , <code>type</code> , <code>payload</code> (json), <code>status</code> , <code>confidence</code> , <code>onec_ref</code> , <code>error</code> |
| <code>mapping_rules</code> | <code>id</code> , <code>company_id</code> , <code>priority</code> , <code>conditions</code> (json), <code>account</code> , <code>item_article</code> , <code>created_from_item</code>                                                    |
| <code>audit_runs</code>    | <code>id</code> , <code>company_id</code> , <code>period_from</code> , <code>period_to</code> , <code>findings</code> (json), <code>created_at</code>                                                                                    |
| <code>action_log</code>    | <code>id</code> , <code>company_id</code> , <code>action</code> , <code>item_id</code> , <code>onec_ref</code> , <code>user</code> , <code>created_at</code>                                                                             |

## 11. Security and compliance

The two things that could end the business are a leaked signature key and a wrong entry in a client's books; both are designed out first.

| Risk                 | Rule                                                                                                                                  |
|----------------------|---------------------------------------------------------------------------------------------------------------------------------------|
| E-IMZO key theft     | Keys never leave the PC; signing only through the local E-IMZO client; no key files or passwords in our code, logs or backend         |
| Claude API key abuse | Key only on the backend; the app gets AI through the proxy with its license token                                                     |
| Wrong data in 1C     | Unposted documents only; review step for low confidence; ExternalID against duplicates; full action log in the app and in PlatformLog |
| 1C credentials       | Stored in Windows Credential Manager, never in SQLite or logs                                                                         |
| Backend breach       | No accounting data on the backend; passwords hashed (argon2); secrets in env/secret store; TLS everywhere; daily encrypted DB backups |
| Pirated copies       | Signed license tokens (Ed25519), device binding, server-side seat limits; obfuscation is not relied on                                |
| Fake updates         | Code-signed installers; update feed over HTTPS with SHA-256 checks                                                                    |
| Payment fraud        | Payme/Click webhooks verified by signature and amount; idempotent processing by provider transaction id                               |

**Data localization.** Uzbekistan's personal data law requires personal data of citizens to be stored on servers in Uzbekistan. The backend (users' phones, names, emails) is hosted in an Uzbek data center (e.g. UZINFOCOM or a local cloud). Confirm the exact obligations with a lawyer before launch.

**AI and client data.** AI requests send parts of 1C data to the Claude API outside Uzbekistan. Mitigations: the AI is optional per company and off by default; the terms of service ask for consent; queries send only the rows needed; counterparty names and INNs of individuals can be masked before sending (phase 2).

**Licensing of reused code.** ROCTUP 1c-mcp-toolkit is GPL-3.0; we use it as reference only and write our own extension, so our product stays closed-source.

## 12. Tech stack and repository

One TypeScript monorepo holds all three systems plus the 1C extension, so types shared between the app, backend and admin never drift.

| Layer          | Choice                                                                                                       |
|----------------|--------------------------------------------------------------------------------------------------------------|
| Language       | TypeScript 5 (strict) everywhere except BSL in the 1C extension                                              |
| Monorepo       | pnpm workspaces + Turborepo                                                                                  |
| Desktop        | Electron, electron-vite, React, TanStack Query, Zustand, Tailwind + shadcn/ui                                |
| Desktop native | winax (COM to 1C), better-sqlite3, keytar / safeStorage, electron-updater, electron-builder (NSIS installer) |
| Backend        | Node.js 22 LTS, Fastify, Zod for validation, Drizzle ORM, PostgreSQL 16, BullMQ + Redis for jobs             |
| Admin          | Vite + React, TanStack Router/Query, shadcn/ui, Recharts                                                     |
| Website        | Astro + React islands                                                                                        |
| AI             | Anthropic TypeScript SDK on the backend only                                                                 |
| i18n           | i18next; uz-Latn, ru, en                                                                                     |
| Testing        | Vitest (unit), Playwright (admin, website, Electron), a test 1C base for connector tests                     |
| CI/CD          | GitHub Actions: lint, test, build; signed Windows installer; Docker images for the backend                   |
| Hosting        | Uzbek data center: backend, Postgres, Redis, update server                                                   |

```

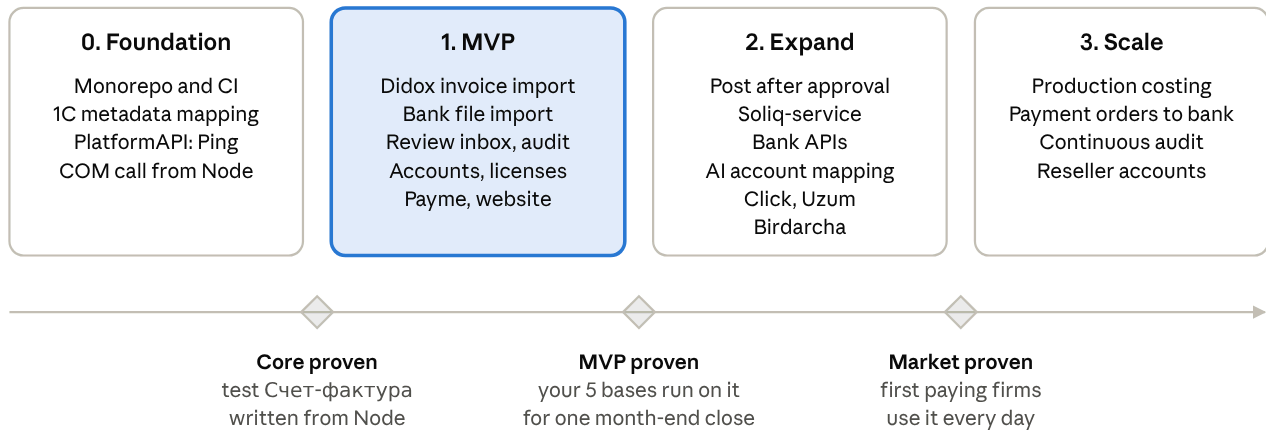
platform/
├─ apps/
│  ├─ desktop/      # Electron: main/, preload/, renderer/
│  ├─ api/          # Fastify backend
│  ├─ admin/        # admin dashboard
│  └─ web/           # marketing site + customer cabinet
├─ packages/
│  ├─ shared/        # types, Zod schemas, API contracts
│  ├─ sources/       # Didox, bank files, soliq files → SourceItem
│  ├─ onec-client/   # COM/HTTP client for PlatformAPI
│  ├─ audit/         # reconciliation rules
│  └─ ui/            # shared React components
├─ onec/
│  └─ extension/     # PlatformAPI extension (XML dump, built to .cfe)
└─ docs/

```

## 13. Roadmap

Coding starts with phase 0: prove our code can write one document into 1C; each phase ends at a gate that must pass before the next begins.

**Each phase ends at a gate that must pass before the next begins**



roadmap · 4 phases, 3 gates

Phases are not to scale; durations will be set once phase 0 shows how fast the 1C write path comes together.

**Phase 0 tasks (start here)**

- ☐ Create the monorepo skeleton (pnpm, Turborepo, TypeScript strict, ESLint, Prettier, Vitest)
- ☐ Read БУ для Узбекистана 3.0 metadata via the MCP setup and write the document field map
- ☐ Build the PlatformAPI extension with Ping and GetOrganizations
- ☐ Call Ping from a Node script through winax on a test copy of a base
- ☐ Add CreateInvoiceReceived and write one unposted Счет-фактура from JSON
- ☐ Electron shell: sign-in screen stub, Companies screen, connector status

## 14. Open questions

These decisions block parts of phase 1; the first four block coding of the connector and imports.

- ☐ Exact metadata of БУ для Узбекистана 3.0: document and field names for Счета-фактуры, bank documents, Контрагенты, Номенклатура, Договоры (read via get\_metadata )
- ☐ Didox API: documentation, partner access, test account, auth method
- ☐ Bank export formats of the banks used by the first five companies
- ☐ COM connection works on the target PC: 64-bit platform, comcntr.dll registered, licenses enough for app + users
- ☐ Which site is "birdarcha" and what data it provides
- ☐ Is there an official API route for my.soliq.uz or Soliq-service data?
- ☐ Product name and domain
- ☐ Plans and prices (per company, per seat, AI quota); trial length
- ☐ Payme merchant account and legal entity for receiving payments
- ☐ Hosting provider in Uzbekistan; lawyer review of personal data and terms of service
- ☐ Code-signing certificate for the Windows installer